

Project 02: Student Management System

Difficulty: Beginner | Estimated Time: 1 Week

1. Project Overview and Goals

The Student Management System (SMS) models a university or college academic database. It covers student enrollment, course management, instructor assignments, grades, attendance, and academic reporting. This project reinforces relational data modeling in a domain that every developer understands intuitively.

Goals:

- Practice many-to-many relationships (students-courses, courses-instructors).
 - Implement GPA calculation logic inside PostgreSQL.
 - Write queries for academic reporting (transcripts, honor rolls, failing students).
 - Use sequences, domains, and custom types.
 - Understand the difference between operational and historical data storage.
-

2. Learning Objectives

By completing this project, you will be able to:

- Model complex many-to-many relationships through junction tables.
 - Use `GENERATED ALWAYS AS` computed columns.
 - Create and use custom `DOMAIN` types for data validation.
 - Write recursive CTEs for hierarchical department data.
 - Implement multi-step business logic in PL/pgSQL functions.
 - Use `CASE` expressions and conditional aggregation for pivot-style reports.
 - Practice `COALESCE`, `NULLIF`, and `GREATEST / LEAST` in calculations.
 - Understand academic GPA calculation as a weighted average.
-

3. Functional Requirements

- **Departments:** Hierarchical departments (School > Department > Program).
 - **Students:** Register students, track academic standing, GPA, and enrollment status.
 - **Instructors:** Manage instructors linked to departments.
 - **Courses:** Define courses with credit hours, prerequisites, and capacity.
 - **Semesters:** Organize academic activity into semesters/terms.
 - **Enrollment:** Enroll students into course sections, track grades.
 - **Attendance:** Record attendance per class session.
 - **Transcripts:** Generate full student transcripts with GPA.
 - **Honor Roll:** Identify students with GPA ≥ 3.5 per semester.
-

4. Non-Functional Requirements

- GPA stored as `NUMERIC(3,2)` and recalculated on every grade update.
- Letter grades must be one of: A, A-, B+, B, B-, C+, C, C-, D+, D, F, I (Incomplete), W (Withdrawn).
- Credit hours must be between 1 and 6.
- A student cannot enroll in the same course in the same semester twice.

- Max section capacity enforced at enrollment time.
- All timestamps use `TIMESTAMPTZ`.

5. Complete Database Schema

```
-- =====
-- STUDENT MANAGEMENT SYSTEM - Complete Schema
-- PostgreSQL 15+
-- =====

CREATE SCHEMA IF NOT EXISTS sms;
SET search_path = sms, public;

-- Custom domain for letter grades
CREATE DOMAIN grade_letter AS VARCHAR(2)
    CHECK (VALUE IN ('A','A-','B+','B','B-','C+','C','C-','D+','D','F','I','W','P','NP'));

-- Custom domain for email
CREATE DOMAIN email_address AS VARCHAR(300)
    CHECK (VALUE ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$');

-----
-- DEPARTMENTS (hierarchical)
-----

CREATE TABLE departments (
    dept_id      SERIAL PRIMARY KEY,
    code         VARCHAR(10) NOT NULL UNIQUE,
    name         VARCHAR(200) NOT NULL,
    parent_id    INTEGER REFERENCES departments(dept_id),
    dean_name    VARCHAR(200),
    established   INTEGER CHECK (established BETWEEN 1800 AND 2100),
    is_active    BOOLEAN NOT NULL DEFAULT TRUE
);

COMMENT ON TABLE departments IS 'Hierarchical department structure';
CREATE INDEX idx_dept_parent ON departments (parent_id);

INSERT INTO departments (code, name, established) VALUES
    ('UNIV', 'University', 1900),
    ('ENG', 'College of Engineering', 1920),
    ('SCI', 'College of Science', 1920),
    ('BUS', 'College of Business', 1930),
    ('ART', 'College of Arts', 1925);

INSERT INTO departments (code, name, parent_id) VALUES
    ('CS', 'Computer Science', 2),
    ('EE', 'Electrical Engineering', 2),
    ('MATH', 'Mathematics', 3),
    ('PHYS', 'Physics', 3),
```

```

('MBA', 'Business Administration',4);

-- -----
-- INSTRUCTORS
-- -----

CREATE TABLE instructors (
    instructor_id    SERIAL PRIMARY KEY,
    employee_id      VARCHAR(20) NOT NULL UNIQUE,
    first_name       VARCHAR(100) NOT NULL,
    last_name        VARCHAR(100) NOT NULL,
    email            email_address NOT NULL UNIQUE,
    dept_id          INTEGER NOT NULL REFERENCES departments(dept_id),
    title            VARCHAR(50) DEFAULT 'Professor'
                    CHECK (title IN ('Professor','Associate Professor','Assistant
Professor',
                                   'Lecturer','Adjunct','Teaching Assistant')),
    phone            VARCHAR(20),
    office_location  VARCHAR(100),
    hire_date        DATE NOT NULL DEFAULT CURRENT_DATE,
    is_active        BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE INDEX idx_instructors_dept ON instructors (dept_id);

-- -----
-- COURSES
-- -----

CREATE TABLE courses (
    course_id        SERIAL PRIMARY KEY,
    course_code      VARCHAR(20) NOT NULL UNIQUE, -- e.g., CS-301
    title            VARCHAR(300) NOT NULL,
    dept_id          INTEGER NOT NULL REFERENCES departments(dept_id),
    credit_hours     SMALLINT NOT NULL DEFAULT 3
                    CHECK (credit_hours BETWEEN 1 AND 6),
    description       TEXT,
    is_elective      BOOLEAN NOT NULL DEFAULT FALSE,
    is_active        BOOLEAN NOT NULL DEFAULT TRUE,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_courses_dept ON courses (dept_id);
CREATE INDEX idx_courses_code ON courses (course_code);

-- -----
-- COURSE PREREQUISITES
-- -----

CREATE TABLE course_prerequisites (
    course_id        INTEGER NOT NULL REFERENCES courses(course_id),
    prereq_id        INTEGER NOT NULL REFERENCES courses(course_id),
    min_grade        grade_letter NOT NULL DEFAULT 'C',
    PRIMARY KEY (course_id, prereq_id),
    CONSTRAINT chk_no_self_prereq CHECK (course_id <> prereq_id)
);

```

```

);

-- -----
-- SEMESTERS / TERMS
-- -----

CREATE TABLE semesters (
    semester_id SERIAL PRIMARY KEY,
    code        VARCHAR(20) NOT NULL UNIQUE,    -- e.g., 2024-FALL
    name        VARCHAR(100) NOT NULL,
    start_date  DATE NOT NULL,
    end_date    DATE NOT NULL,
    is_current  BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT chk_semester_dates CHECK (end_date > start_date)
);

-- Enforce only one current semester
CREATE UNIQUE INDEX idx_semesters_current
    ON semesters (is_current)
    WHERE is_current = TRUE;

INSERT INTO semesters (code, name, start_date, end_date, is_current) VALUES
    ('2023-FALL', 'Fall 2023', '2023-08-28', '2023-12-15', FALSE),
    ('2024-SPRING', 'Spring 2024', '2024-01-15', '2024-05-10', FALSE),
    ('2024-FALL', 'Fall 2024', '2024-08-26', '2024-12-13', TRUE);

-- -----
-- COURSE SECTIONS
-- -----

CREATE TABLE sections (
    section_id SERIAL PRIMARY KEY,
    course_id  INTEGER NOT NULL REFERENCES courses(course_id),
    semester_id INTEGER NOT NULL REFERENCES semesters(semester_id),
    instructor_id INTEGER NOT NULL REFERENCES instructors(instructor_id),
    section_number VARCHAR(10) NOT NULL DEFAULT '001',
    room        VARCHAR(50),
    schedule    VARCHAR(200),    -- e.g., MWF 09:00-10:00
    capacity    SMALLINT NOT NULL DEFAULT 30 CHECK (capacity > 0),
    enrolled_count SMALLINT NOT NULL DEFAULT 0 CHECK (enrolled_count >= 0),
    status      VARCHAR(20) NOT NULL DEFAULT 'Open'
                CHECK (status IN ('Open', 'Full', 'Cancelled', 'Completed')),
    UNIQUE (course_id, semester_id, section_number)
);

CREATE INDEX idx_sections_course    ON sections (course_id);
CREATE INDEX idx_sections_semester  ON sections (semester_id);
CREATE INDEX idx_sections_instructor ON sections (instructor_id);

-- -----
-- STUDENTS
-- -----

CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,

```

```

    student_number    VARCHAR(20) NOT NULL UNIQUE, -- e.g., S20240001
    first_name        VARCHAR(100) NOT NULL,
    last_name         VARCHAR(100) NOT NULL,
    email             email_address NOT NULL UNIQUE,
    phone             VARCHAR(20),
    date_of_birth      DATE,
    gender            VARCHAR(20) CHECK (gender IN ('Male','Female','Non-
binary','Prefer not to say')),
    dept_id           INTEGER REFERENCES departments(dept_id),
    major            VARCHAR(200),
    enrollment_year    SMALLINT NOT NULL DEFAULT EXTRACT(YEAR FROM NOW())::SMALLINT,
    enrollment_status VARCHAR(20) NOT NULL DEFAULT 'Active'
                    CHECK (enrollment_status IN
('Active','Inactive','Graduated','Suspended','Withdrawn')),
    cumulative_gpa     NUMERIC(3,2) DEFAULT 0.00 CHECK (cumulative_gpa BETWEEN 0.00
AND 4.00),
    total_credits      SMALLINT NOT NULL DEFAULT 0,
    address           TEXT,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_students_number ON students (student_number);
CREATE INDEX idx_students_email  ON students (email);
CREATE INDEX idx_students_dept   ON students (dept_id);
CREATE INDEX idx_students_status ON students (enrollment_status);

-----
-- ENROLLMENTS
-----

CREATE TABLE enrollments (
    enrollment_id SERIAL PRIMARY KEY,
    student_id    INTEGER NOT NULL REFERENCES students(student_id),
    section_id    INTEGER NOT NULL REFERENCES sections(section_id),
    enrolled_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    status        VARCHAR(20) NOT NULL DEFAULT 'Enrolled'
                CHECK (status IN
('Enrolled','Completed','Withdrawn','Incomplete')),
    grade         grade_letter,
    grade_points  NUMERIC(3,2), -- 4.0 scale numeric equivalent
    UNIQUE (student_id, section_id)
);

CREATE INDEX idx_enrollments_student ON enrollments (student_id);
CREATE INDEX idx_enrollments_section ON enrollments (section_id);
CREATE INDEX idx_enrollments_grade   ON enrollments (grade);

-- Grade point mapping
CREATE TABLE grade_points_map (
    grade         grade_letter PRIMARY KEY,
    points        NUMERIC(3,2) NOT NULL,
    description    VARCHAR(50)
);

```

```

INSERT INTO grade_points_map (grade, points, description) VALUES
  ('A', 4.00, 'Excellent'),
  ('A-', 3.70, 'Excellent-'),
  ('B+', 3.30, 'Above Average'),
  ('B', 3.00, 'Good'),
  ('B-', 2.70, 'Good-'),
  ('C+', 2.30, 'Above Average'),
  ('C', 2.00, 'Satisfactory'),
  ('C-', 1.70, 'Satisfactory-'),
  ('D+', 1.30, 'Below Average'),
  ('D', 1.00, 'Poor'),
  ('F', 0.00, 'Failing'),
  ('I', NULL, 'Incomplete'),
  ('W', NULL, 'Withdrawn'),
  ('P', NULL, 'Pass'),
  ('NP', NULL, 'No Pass');

-----
-- ATTENDANCE
-----

CREATE TABLE class_sessions (
  session_id SERIAL PRIMARY KEY,
  section_id INTEGER NOT NULL REFERENCES sections(section_id),
  session_date DATE NOT NULL,
  topic VARCHAR(300),
  UNIQUE (section_id, session_date)
);

CREATE TABLE attendance (
  attendance_id SERIAL PRIMARY KEY,
  session_id INTEGER NOT NULL REFERENCES class_sessions(session_id),
  student_id INTEGER NOT NULL REFERENCES students(student_id),
  status VARCHAR(20) NOT NULL DEFAULT 'Present'
  CHECK (status IN ('Present', 'Absent', 'Late', 'Excused')),
  notes TEXT,
  UNIQUE (session_id, student_id)
);

CREATE INDEX idx_attendance_student ON attendance (student_id);
CREATE INDEX idx_attendance_session ON attendance (session_id);

```

6. ASCII ER Diagram

```

DEPARTMENTS (hierarchical)
+-----+
| dept_id (PK) |<--+
| code (UNIQUE) |  | parent_id (self-ref)
| name        |---+
| parent_id (FK) |

```

```

+-----+-----+
      |         \
      |         \
      v         v
    INSTRUCTORS    COURSES
+-----+-----+ +-----+-----+
| instructor_id| | course_id (PK) |
| employee_id  | | course_code   |
| first_name   | | title         |
| dept_id (FK) | | dept_id (FK)   |
| title        | | credit_hours  |
+-----+-----+ +-----+-----+
      |         |
      |         | --- COURSE_PREREQUISITES
      v         | +-----+-----+
    SECTIONS     | | course_id (FK) |
+-----+-----+ | | prereq_id (FK) |
| section_id |<-----+ +-----+-----+
| course_id  |
| semester_id|<-- SEMESTERS
| instructor_id| +-----+-----+
| capacity    | | semester_id|
| enrolled_cnt| | code        |
+-----+-----+ | start_date |
      |         | is_current |
      |         +-----+-----+
      v
    ENROLLMENTS

+-----+-----+ STUDENTS
| enrollment_id | +-----+-----+
| student_id (FK)|-->| student_id (PK) |
| section_id (FK)| | student_number |
| grade         | | first_name   |
| grade_points  | | cumulative_gpa |
+-----+-----+ | dept_id (FK) |
      |         | enrollment_status |
      v         +-----+-----+
    CLASS_SESSIONS      |
+-----+-----+      |
| session_id   |      v
| section_id   |      ATTENDANCE
| session_date | +-----+-----+
+-----+-----+ | attendance_id |
      |         | session_id (FK) |
      +----->| student_id (FK) |
      |         | status         |
      +-----+-----+

GRADE_POINTS_MAP
+-----+-----+
| grade (PK)   |
| points       |

```

```
| description      |  
+-----+
```

7. Sample Data Scripts

```
SET search_path = sms, public;  
  
-- Instructors  
INSERT INTO instructors (employee_id, first_name, last_name, email, dept_id, title,  
office_location) VALUES  
('EMP001', 'Dr. Alan', 'Turing', 'a.turing@univ.edu', 6, 'Professor',  
'CS-201'),  
('EMP002', 'Dr. Grace', 'Hopper', 'g.hopper@univ.edu', 6, 'Associate  
Professor', 'CS-202'),  
('EMP003', 'Dr. Marie', 'Curie', 'm.curie@univ.edu', 8, 'Professor',  
'SCI-101'),  
('EMP004', 'Dr. John', 'von Neumann', 'j.vneumann@univ.edu', 6, 'Professor',  
'CS-203'),  
('EMP005', 'Prof. Ada', 'Lovelace', 'a.lovelace@univ.edu', 8, 'Lecturer',  
'SCI-105');  
  
-- Courses  
INSERT INTO courses (course_code, title, dept_id, credit_hours, is_elective) VALUES  
('CS-101', 'Introduction to Programming', 6, 3, FALSE),  
('CS-201', 'Data Structures', 6, 3, FALSE),  
('CS-301', 'Database Systems', 6, 3, FALSE),  
('CS-401', 'Advanced Algorithms', 6, 3, FALSE),  
('MATH-101', 'Calculus I', 8, 4, FALSE),  
('MATH-201', 'Linear Algebra', 8, 3, FALSE),  
('CS-350', 'Machine Learning', 6, 3, TRUE),  
('CS-410', 'Cloud Computing', 6, 3, TRUE);  
  
-- Prerequisites  
INSERT INTO course_prerequisites (course_id, prereq_id, min_grade) VALUES  
(2, 1, 'C'), -- CS-201 requires CS-101  
(3, 2, 'C'), -- CS-301 requires CS-201  
(4, 2, 'B'), -- CS-401 requires CS-201 with B  
(7, 3, 'C'); -- ML requires DB Systems  
  
-- Sections (Fall 2024 = semester_id 3)  
INSERT INTO sections (course_id, semester_id, instructor_id, section_number, room,  
schedule, capacity) VALUES  
(1, 3, 1, '001', 'CS-Hall-101', 'MWF 09:00-10:00', 40),  
(2, 3, 2, '001', 'CS-Hall-202', 'TTH 10:00-11:30', 35),  
(3, 3, 1, '001', 'CS-Hall-305', 'MWF 11:00-12:00', 30),  
(5, 3, 3, '001', 'SCI-Hall-210', 'MWF 08:00-09:00', 50),  
(7, 3, 4, '001', 'CS-Hall-401', 'TTH 13:00-14:30', 25);  
  
-- Students  
INSERT INTO students (student_number, first_name, last_name, email, dept_id, major,
```



```

enrollment_year) VALUES
  ('S20200001', 'Alice', 'Chen', 'alice.chen@student.edu', 6, 'Computer
Science', 2020),
  ('S20200002', 'Bob', 'Patel', 'bob.patel@student.edu', 6, 'Computer
Science', 2020),
  ('S20210001', 'Carol', 'Williams', 'carol.w@student.edu', 8,
'Mathematics', 2021),
  ('S20210002', 'David', 'Kim', 'david.kim@student.edu', 6, 'Computer
Science', 2021),
  ('S20220001', 'Emma', 'Garcia', 'emma.g@student.edu', 6, 'Computer
Science', 2022),
  ('S20220002', 'Frank', 'Nguyen', 'frank.n@student.edu', 6, 'Computer
Science', 2022),
  ('S20230001', 'Grace', 'Lee', 'grace.lee@student.edu', 6, 'Computer
Science', 2023),
  ('S20230002', 'Henry', 'Okonkwo', 'henry.o@student.edu', 8, 'Physics',
2023);

-- Enrollments (current semester)
INSERT INTO enrollments (student_id, section_id, grade, grade_points, status) VALUES
  (1, 3, 'A', 4.00, 'Completed'),
  (2, 3, 'B+', 3.30, 'Completed'),
  (3, 4, 'A-', 3.70, 'Completed'),
  (4, 2, 'B', 3.00, 'Completed'),
  (5, 1, 'C+', 2.30, 'Enrolled'),
  (6, 1, 'B-', 2.70, 'Enrolled'),
  (7, 1, NULL, NULL, 'Enrolled'),
  (8, 4, 'A', 4.00, 'Completed'),
  (1, 5, 'A-', 3.70, 'Completed'),
  (4, 3, 'C', 2.00, 'Completed');

-- Update enrolled counts
UPDATE sections SET enrolled_count = (
  SELECT COUNT(*) FROM enrollments
  WHERE section_id = sections.section_id AND status = 'Enrolled'
);

```

8. Core SQL Queries

```

SET search_path = sms, public;

-- -----
-- Q1: Student transcript - all courses and grades
-- -----

SELECT
  s.student_number,
  s.first_name || ' ' || s.last_name AS student,
  sem.name AS semester,
  c.course_code,
  c.title,

```

```

        c.credit_hours,
        e.grade,
        e.grade_points,
        e.status
FROM enrollments e
JOIN students s    ON e.student_id = s.student_id
JOIN sections sec  ON e.section_id = sec.section_id
JOIN courses c     ON sec.course_id = c.course_id
JOIN semesters sem ON sec.semester_id = sem.semester_id
WHERE s.student_id = 1
ORDER BY sem.start_date, c.course_code;

-- -----
-- Q2: Calculate GPA per student per semester (weighted avg)
-- -----
SELECT
    s.student_number,
    s.first_name || ' ' || s.last_name AS student,
    sem.name                          AS semester,
    SUM(c.credit_hours * e.grade_points)
      / NULLIF(SUM(c.credit_hours) FILTER (WHERE e.grade_points IS NOT NULL), 0)
      AS semester_gpa,
    SUM(c.credit_hours) FILTER (WHERE e.grade NOT IN ('I','W') AND e.grade IS NOT
NULL)
      AS credits_earned
FROM enrollments e
JOIN students s    ON e.student_id = s.student_id
JOIN sections sec  ON e.section_id = sec.section_id
JOIN courses c     ON sec.course_id = c.course_id
JOIN semesters sem ON sec.semester_id = sem.semester_id
WHERE e.status = 'Completed'
GROUP BY s.student_id, s.student_number, s.first_name, s.last_name, sem.semester_id,
sem.name
ORDER BY s.last_name, sem.start_date;

-- -----
-- Q3: Honor Roll (GPA >= 3.5) for current semester
-- -----
WITH semester_gpa AS (
    SELECT
        e.student_id,
        ROUND(SUM(c.credit_hours * e.grade_points)
          / NULLIF(SUM(c.credit_hours) FILTER (WHERE e.grade_points IS NOT NULL),
0), 2)
          AS gpa
    FROM enrollments e
    JOIN sections sec  ON e.section_id = sec.section_id
    JOIN courses c     ON sec.course_id = c.course_id
    JOIN semesters sem ON sec.semester_id = sem.semester_id
    WHERE sem.is_current = TRUE AND e.status = 'Completed'
    GROUP BY e.student_id
)

```

```

SELECT
    s.student_number,
    s.first_name || ' ' || s.last_name AS student,
    g.gpa,
    CASE WHEN g.gpa >= 3.9 THEN 'Summa Cum Laude'
         WHEN g.gpa >= 3.7 THEN 'Magna Cum Laude'
         WHEN g.gpa >= 3.5 THEN 'Cum Laude'
    END AS honor_distinction
FROM semester_gpa g
JOIN students s ON g.student_id = s.student_id
WHERE g.gpa >= 3.5
ORDER BY g.gpa DESC;

```

```

-- -----
-- Q4: Students at academic risk (GPA < 2.0)
-- -----

```

```

SELECT
    s.student_number,
    s.first_name || ' ' || s.last_name AS student,
    s.major,
    s.cumulative_gpa,
    s.total_credits
FROM students s
WHERE s.cumulative_gpa < 2.0
     AND s.enrollment_status = 'Active'
ORDER BY s.cumulative_gpa;

```

```

-- -----
-- Q5: Course enrollment summary with fill rate
-- -----

```

```

SELECT
    c.course_code,
    c.title,
    sec.section_number,
    i.first_name || ' ' || i.last_name AS instructor,
    sec.capacity,
    sec.enrolled_count,
    ROUND(100.0 * sec.enrolled_count / sec.capacity, 1) AS fill_pct,
    sec.room,
    sec.schedule
FROM sections sec
JOIN courses c ON sec.course_id = c.course_id
JOIN instructors i ON sec.instructor_id = i.instructor_id
JOIN semesters sem ON sec.semester_id = sem.semester_id
WHERE sem.is_current = TRUE
ORDER BY fill_pct DESC;

```

```

-- -----
-- Q6: Grade distribution for a course
-- -----

```

```

SELECT
    c.course_code,

```

```

        c.title,
        e.grade,
        COUNT(*) AS count,
        ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (PARTITION BY c.course_id), 1) AS
pct
FROM enrollments e
JOIN sections sec ON e.section_id = sec.section_id
JOIN courses c ON sec.course_id = c.course_id
WHERE c.course_code = 'CS-301'
      AND e.grade IS NOT NULL
GROUP BY c.course_id, c.course_code, c.title, e.grade
ORDER BY e.grade;

```

```

-- -----
-- Q7: Instructor workload summary
-- -----

```

```

SELECT
    i.first_name || ' ' || i.last_name AS instructor,
    i.title,
    d.name AS department,
    COUNT(DISTINCT sec.section_id) AS sections_teaching,
    SUM(sec.enrolled_count) AS total_students,
    SUM(c.credit_hours) AS total_credit_hours
FROM sections sec
JOIN instructors i ON sec.instructor_id = i.instructor_id
JOIN courses c ON sec.course_id = c.course_id
JOIN departments d ON i.dept_id = d.dept_id
JOIN semesters sem ON sec.semester_id = sem.semester_id
WHERE sem.is_current = TRUE
GROUP BY i.instructor_id, i.first_name, i.last_name, i.title, d.name
ORDER BY total_students DESC;

```

```

-- -----
-- Q8: Check prerequisite eligibility for a student
-- -----

```

```

WITH completed_courses AS (
    SELECT c.course_id, e.grade, gpm.points
    FROM enrollments e
    JOIN sections sec ON e.section_id = sec.section_id
    JOIN courses c ON sec.course_id = c.course_id
    JOIN grade_points_map gpm ON e.grade = gpm.grade
    WHERE e.student_id = 2 AND e.status = 'Completed'
),
prereqs AS (
    SELECT cp.course_id, cp.prereq_id, cp.min_grade
    FROM course_prerequisites cp
    WHERE cp.course_id = 4 -- CS-401
)
SELECT
    c.course_code AS required_prereq,
    p.min_grade AS minimum_grade,
    cc.grade AS student_grade,

```

```

CASE WHEN cc.course_id IS NOT NULL
      AND gpm.points >= gpm_min.points THEN 'MET'
      WHEN cc.course_id IS NULL THEN 'NOT TAKEN'
      ELSE 'GRADE TOO LOW'
END AS prereq_status
FROM prereqs p
JOIN courses c          ON p.prereq_id = c.course_id
LEFT JOIN completed_courses cc  ON p.prereq_id = cc.course_id
LEFT JOIN grade_points_map gpm  ON cc.grade = gpm.grade
LEFT JOIN grade_points_map gpm_min ON p.min_grade = gpm_min.grade;

-----
-- Q9: Attendance rate per student per section
-----
SELECT
  s.first_name || ' ' || s.last_name AS student,
  c.course_code,
  COUNT(cs.session_id)                AS total_classes,
  COUNT(a.attendance_id) FILTER (WHERE a.status = 'Present') AS present,
  COUNT(a.attendance_id) FILTER (WHERE a.status = 'Absent')  AS absent,
  ROUND(100.0 * COUNT(a.attendance_id) FILTER (WHERE a.status = 'Present')
        / NULLIF(COUNT(cs.session_id), 0), 1) AS attendance_pct
FROM class_sessions cs
JOIN sections sec  ON cs.section_id = sec.section_id
JOIN courses c    ON sec.course_id = c.course_id
JOIN enrollments e ON sec.section_id = e.section_id
JOIN students s   ON e.student_id = s.student_id
LEFT JOIN attendance a ON cs.session_id = a.session_id
                  AND s.student_id = a.student_id
GROUP BY s.student_id, s.first_name, s.last_name, c.course_code
ORDER BY attendance_pct;

-----
-- Q10: Recursive CTE - Department hierarchy
-----
WITH RECURSIVE dept_tree AS (
  -- Base: top-level departments
  SELECT dept_id, code, name, parent_id, 0 AS depth,
         name::TEXT AS path
  FROM departments WHERE parent_id IS NULL

  UNION ALL

  SELECT d.dept_id, d.code, d.name, d.parent_id,
         dt.depth + 1,
         dt.path || ' > ' || d.name
  FROM departments d
  JOIN dept_tree dt ON d.parent_id = dt.dept_id
)
SELECT REPEAT(' ', depth) || code AS indented_code, name, path
FROM dept_tree
ORDER BY path;

```

```

-- -----
-- Q11: Courses students are eligible to take next semester
-- -----

WITH earned AS (
    SELECT DISTINCT ON (c.course_id)
        c.course_id, e.grade
    FROM enrollments e
    JOIN sections sec ON e.section_id = sec.section_id
    JOIN courses c    ON sec.course_id = c.course_id
    WHERE e.student_id = 1 AND e.status = 'Completed'
    ORDER BY c.course_id, e.grade DESC
)
SELECT c.course_code, c.title, c.credit_hours
FROM courses c
WHERE c.is_active = TRUE
    AND c.course_id NOT IN (SELECT course_id FROM earned)
    AND NOT EXISTS (
        SELECT 1 FROM course_prerequisites cp
        LEFT JOIN earned e ON cp.prereq_id = e.course_id
        LEFT JOIN grade_points_map gpm_st ON e.grade = gpm_st.grade
        LEFT JOIN grade_points_map gpm_min ON cp.min_grade = gpm_min.grade
        WHERE cp.course_id = c.course_id
            AND (e.course_id IS NULL OR gpm_st.points < gpm_min.points)
    )
ORDER BY c.course_code;

-- -----
-- Q12: Year-over-year enrollment comparison
-- -----

SELECT
    sem.name AS semester,
    COUNT(DISTINCT e.student_id) AS enrolled_students,
    COUNT(e.enrollment_id) AS total_enrollments,
    ROUND(AVG(e.grade_points) FILTER (WHERE e.grade_points IS NOT NULL), 2) AS
avg_gpa
FROM semesters sem
LEFT JOIN sections sec ON sec.semester_id = sem.semester_id
LEFT JOIN enrollments e ON e.section_id = sec.section_id
GROUP BY sem.semester_id, sem.name, sem.start_date
ORDER BY sem.start_date;

-- -----
-- Q13: Students with incomplete grades
-- -----

SELECT
    s.student_number,
    s.first_name || ' ' || s.last_name AS student,
    c.course_code,
    c.title,
    sem.name AS semester,
    i.first_name || ' ' || i.last_name AS instructor

```

```

FROM enrollments e
JOIN students s    ON e.student_id = s.student_id
JOIN sections sec  ON e.section_id = sec.section_id
JOIN courses c     ON sec.course_id = c.course_id
JOIN semesters sem ON sec.semester_id = sem.semester_id
JOIN instructors i ON sec.instructor_id = i.instructor_id
WHERE e.grade = 'I'
ORDER BY sem.start_date, s.last_name;

-- -----
-- Q14: Department performance summary
-- -----

SELECT
    d.name                                AS department,
    COUNT(DISTINCT s.student_id)         AS students,
    ROUND(AVG(s.cumulative_gpa), 2)      AS avg_gpa,
    COUNT(DISTINCT c.course_id)          AS courses_offered,
    COUNT(DISTINCT i.instructor_id)      AS instructors
FROM departments d
LEFT JOIN students s    ON d.dept_id = s.dept_id AND s.enrollment_status = 'Active'
LEFT JOIN courses c     ON d.dept_id = c.dept_id AND c.is_active = TRUE
LEFT JOIN instructors i ON d.dept_id = i.dept_id AND i.is_active = TRUE
WHERE d.parent_id IS NOT NULL
GROUP BY d.dept_id, d.name
ORDER BY avg_gpa DESC NULLS LAST;

-- -----
-- Q15: Grade pivot - count of each grade per course
-- -----

SELECT
    c.course_code,
    COUNT(*) FILTER (WHERE e.grade = 'A') AS grade_A,
    COUNT(*) FILTER (WHERE e.grade = 'B') AS grade_B,
    COUNT(*) FILTER (WHERE e.grade = 'C') AS grade_C,
    COUNT(*) FILTER (WHERE e.grade = 'D') AS grade_D,
    COUNT(*) FILTER (WHERE e.grade = 'F') AS grade_F,
    COUNT(*) FILTER (WHERE e.grade = 'W') AS grade_W,
    ROUND(AVG(e.grade_points), 2)         AS class_gpa_avg
FROM enrollments e
JOIN sections sec ON e.section_id = sec.section_id
JOIN courses c   ON sec.course_id = c.course_id
GROUP BY c.course_id, c.course_code
ORDER BY c.course_code;

```

9. Stored Procedures / Functions

```

-- -----
-- FUNCTION 1: Calculate and update cumulative GPA for a student
-- -----

CREATE OR REPLACE FUNCTION sms.calculate_cumulative_gpa(p_student_id INTEGER)

```

```

RETURNS NUMERIC(3,2)
LANGUAGE plpgsql AS $$
DECLARE
    v_gpa          NUMERIC(3,2);
    v_total_credits SMALLINT;
BEGIN
    SELECT
        ROUND(
            SUM(c.credit_hours * e.grade_points)::NUMERIC
            / NULLIF(SUM(c.credit_hours) FILTER (WHERE e.grade_points IS NOT NULL),
0),
        2),
        COALESCE(SUM(c.credit_hours) FILTER (WHERE e.grade NOT IN ('I','W','F') AND
e.grade IS NOT NULL), 0)
    INTO v_gpa, v_total_credits
    FROM enrollments e
    JOIN sections sec ON e.section_id = sec.section_id
    JOIN courses c    ON sec.course_id = c.course_id
    WHERE e.student_id = p_student_id
        AND e.status = 'Completed';

    UPDATE sms.students
    SET cumulative_gpa = COALESCE(v_gpa, 0.00),
        total_credits = COALESCE(v_total_credits, 0)
    WHERE student_id = p_student_id;

    RETURN COALESCE(v_gpa, 0.00);
END;
$$;

-- -----
-- FUNCTION 2: Enroll student in a section with all validations
-- -----

CREATE OR REPLACE FUNCTION sms.enroll_student(
    p_student_id INTEGER,
    p_section_id INTEGER
)
RETURNS TEXT
LANGUAGE plpgsql AS $$
DECLARE
    v_student sms.students%ROWTYPE;
    v_section sms.sections%ROWTYPE;
    v_course_id INTEGER;
BEGIN
    SELECT * INTO v_student FROM sms.students WHERE student_id = p_student_id;
    IF NOT FOUND THEN RAISE EXCEPTION 'Student % not found', p_student_id; END IF;
    IF v_student.enrollment_status <> 'Active' THEN
        RAISE EXCEPTION 'Student % is not in Active status', p_student_id;
    END IF;

    SELECT * INTO v_section FROM sms.sections WHERE section_id = p_section_id FOR
UPDATE;

```



```

IF NOT FOUND THEN RAISE EXCEPTION 'Section % not found', p_section_id; END IF;
IF v_section.status = 'Full' THEN
    RAISE EXCEPTION 'Section % is full', p_section_id;
END IF;
IF v_section.status = 'Cancelled' THEN
    RAISE EXCEPTION 'Section % has been cancelled', p_section_id;
END IF;

SELECT course_id INTO v_course_id FROM sms.sections WHERE section_id =
p_section_id;

-- Check if already enrolled in this course this semester
IF EXISTS (
    SELECT 1 FROM sms.enrollments e
    JOIN sms.sections s ON e.section_id = s.section_id
    WHERE e.student_id = p_student_id
        AND s.course_id = v_course_id
        AND s.semester_id = v_section.semester_id
        AND e.status = 'Enrolled'
) THEN
    RAISE EXCEPTION 'Student % is already enrolled in this course for this
semester', p_student_id;
END IF;

INSERT INTO sms.enrollments (student_id, section_id, status)
VALUES (p_student_id, p_section_id, 'Enrolled');

UPDATE sms.sections
SET enrolled_count = enrolled_count + 1,
    status = CASE WHEN enrolled_count + 1 >= capacity THEN 'Full' ELSE status
END
WHERE section_id = p_section_id;

RETURN format('Student %s enrolled in section %s successfully', p_student_id,
p_section_id);
END;
$$;

-----
-- FUNCTION 3: Post a grade and recalculate GPA
-----

CREATE OR REPLACE FUNCTION sms.post_grade(
    p_enrollment_id INTEGER,
    p_grade          grade_letter
)
RETURNS TEXT
LANGUAGE plpgsql AS $$
DECLARE
    v_enroll    sms.enrollments%ROWTYPE;
    v_points    NUMERIC(3,2);
BEGIN
    SELECT * INTO v_enroll FROM sms.enrollments WHERE enrollment_id =

```

```

p_enrollment_id FOR UPDATE;
    IF NOT FOUND THEN RAISE EXCEPTION 'Enrollment % not found', p_enrollment_id; END
IF;

    SELECT points INTO v_points FROM sms.grade_points_map WHERE grade = p_grade;

    UPDATE sms.enrollments
    SET grade = p_grade,
        grade_points = v_points,
        status = CASE WHEN p_grade IN ('I','W') THEN p_grade::TEXT
                        ELSE 'Completed' END
    WHERE enrollment_id = p_enrollment_id;

    PERFORM sms.calculate_cumulative_gpa(v_enroll.student_id);

    RETURN format('Grade %s posted. GPA recalculated.', p_grade);
END;
$$;

```

10. Triggers

```

-- -----
-- TRIGGER 1: Validate section capacity before enrollment
-- -----

CREATE OR REPLACE FUNCTION sms.trg_check_capacity()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_section sms.sections%ROWTYPE;
BEGIN
    SELECT * INTO v_section FROM sms.sections WHERE section_id = NEW.section_id;
    IF v_section.enrolled_count >= v_section.capacity THEN
        RAISE EXCEPTION 'Cannot enroll: section % is at full capacity (%)',
            NEW.section_id, v_section.capacity;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_enrollment_capacity
    BEFORE INSERT ON sms.enrollments
    FOR EACH ROW
    EXECUTE FUNCTION sms.trg_check_capacity();

-- -----
-- TRIGGER 2: Auto-recalculate GPA when grade is updated
-- -----

CREATE OR REPLACE FUNCTION sms.trg_recalculate_gpa()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.grade IS DISTINCT FROM OLD.grade THEN

```

```

        PERFORM sms.calculate_cumulative_gpa(NEW.student_id);
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_grade_gpa_update
    AFTER UPDATE OF grade ON sms.enrollments
    FOR EACH ROW
    EXECUTE FUNCTION sms.trg_recalculate_gpa();

```

11. Performance Optimization

Index	Purpose
idx_students_status	Filter active students quickly
idx_enrollments_student	Fast lookup of all courses per student
idx_sections_semester + idx_sections_instructor	Instructor schedule and semester reporting
Partial index on is_current=TRUE for semesters	Ensure single-row enforcement and fast lookup
Composite on (student_id, section_id)	UNIQUE constraint doubles as performance index

```

-- Analyze query plan for transcript lookup
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sms.enrollments
WHERE student_id = 1;

```

12. Extensions Used

```

CREATE EXTENSION IF NOT EXISTS pg_trgm; -- fuzzy student name search
CREATE EXTENSION IF NOT EXISTS tablefunc; -- crosstab for grade pivot reports

```

13. Testing Guide

```

-- TEST 1: Enroll student successfully
SELECT sms.enroll_student(7, 2);

-- TEST 2: Try to enroll in full section
UPDATE sms.sections SET enrolled_count = capacity WHERE section_id = 5;
SELECT sms.enroll_student(8, 5); -- Should fail

-- TEST 3: Post grade and verify GPA recalculation
SELECT sms.post_grade(7, 'B+');
SELECT cumulative_gpa FROM sms.students WHERE student_id = 7;

```

```
-- TEST 4: Honor roll report
-- Manually set a student to high GPA and verify they appear
UPDATE sms.students SET cumulative_gpa = 3.92 WHERE student_id = 1;
-- Run Q3 honor roll query

-- TEST 5: Recursive department hierarchy
-- Run Q10 and verify correct indentation levels

-- TEST 6: Prerequisite check
-- Run Q8 for student 2 on course 4 (CS-401)
```

14. Extension Challenges

- Online Exam Module:** Add tables for exams, questions, student responses, and auto-grading. Implement a function that calculates exam scores and updates the enrollment grade.
- Tuition and Billing:** Add a billing module that calculates tuition per semester based on credit hours and membership type. Track payments and balances.
- Academic Calendar:** Extend semesters with holidays, add-drop deadlines, and final exam schedules. Write a function that checks whether a given operation is within the allowed date window.
- Student Advising:** Add an advisor table (one instructor per student). Create a `student_notes` table for advising sessions. Write reports for advisors showing all their advisees' GPA trends.
- Degree Audit:** Define degree requirements per program (required courses, minimum credits, minimum GPA). Write a function that compares a student's completed courses against requirements and returns what's missing.

15. What You Learned

Skill	Demonstrated By
Custom DOMAIN types	grade_letter, email_address domains
Recursive CTEs	Department hierarchy traversal
Weighted average GPA	Aggregate with conditional filtering
Many-to-many relationships	book_authors, enrollments, course_prerequisites
Conditional aggregation	Grade pivot with COUNT FILTER WHERE
Trigger-driven computed columns	Auto GPA recalculation on grade change
Business rule enforcement	Capacity checks, prerequisite validation
Academic reporting	Honor roll, at-risk students, transcripts